5주차 개념정리

개념정리-BERT

- 트랜스포머 기반 양방향 인코더 사용 자연어 처리 모델

> 양방향(Bidirectional)

🔎 **단방향 vs. 양방향**

- 단방향: 문장의 입력 데이터를 단방향으로 처리함
- 양방향: 입력 데이터를 양방향에서 처리함.
  - 마스킹된 부분의 이전과 이후 단어를 모두 참조하면서 단어의 의미와 문맥을 파악
  - 기존과 달리 양방향에서 학습한다.

BERT는 사전 학습을 위해 MLM과 NSP를 사용한다.

사전 학습 방법

MLM (마스킹된 언어 모델링)

> Masked Language Modeling

- 입력 문장에서 랜덤으로 일부 단어를 마스킹하고 그 단어를 예측하는 방식이다.
- → 학습 결과 누락 단어를 추론하는 능력을 모델이 갖게 된다.

NSP (다음 문장 예측 방법)

- 두 개의 문장이 주어졌을 때, B문장이 A 다음에 이어지는 문장인지 그 여부를 판단하는 작업
- → 학습 결과 두 문장 간 관계를 학습하고 의미적 유사성을 파악할 수 있게 된다.

🔎 **[CLS]** 토큰: 입력 문장의 시작 부분에 추가되는 토큰
[SEP] 토큰: 두 개 이상의 문장을 구분하기 위해 사용되는 토큰
[MASK] 토큰: 입력 문장 내에서 임의로 선택된 단어를 가리키는 토큰

- 텍스트 토큰 중 15%에 해당하는 단어를 마스킹한다.
  - 그 중 80% → `[MASK]`
  - 10% → 랜덤 단어로 변경
  - 10% → 기존 토큰 그대로 두기
- 입력 시퀀스는 `[CLS]` 문장-1 `[SEP]` 문장-2 `[MASK]` `[SEP]` 의 구조를 갖게 된다.

모델 실습

네이버 영화 리뷰 데이터 불러오기

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=4
2)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6*len(df)),
int(0.8*len(df))]
)

print(train.head(5).to_markdown())

print(f"Training Data Size: {len(train)}")
print(f"Validation Data Size: {len(valid)}")
print(f"Testing Data Size: {len(test)}")
```

```
[nsmc] download ratings_train.txt: 14.6MB [00:00, 96.9MB/s]
[nsmc] download ratings_test.txt: 4.90MB [00:00, 63.1MB/s]
|       | text                                                    |   label |
|------:|:------------------------------------------------------|--------:|
| 26891 | 역시 코믹액션은 성룡, 홍금보, 원표 삼인방이 최고지!!     |       1 |
| 25024 | 점수 후하게 줘야것네 별 반개~                            |       0 |
| 11666 | 오랜만에 느낄수 있는 [감독] 구타욕구.                    |       0 |
| 40303 | 본지는 좀 됐지만 극장서 돈주고 본게 아직까지 아까운 영화 |       0 |
| 18010 | 징키스칸이란 소재를 가지고 이것밖에 못만드냐             |       0 |
Training Data Size: 12000
Validation Data Size: 4000
Testing Data Size: 4000
```

BERT 입력 텐서 생성

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=4
2)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6*len(df)),
int(0.8*len(df))]
)

import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampl
er

def make_dataset(data, tokenizer, device):
  tokenized = tokenizer(
      text=data.text.tolist(),
      padding="longest",
      truncation=True,
      return_tensors="pt"
    )
  input_ids = tokenized["input_ids"].to(device)
  attention_mask = tokenized["attention_mask"].to(device)
```

```
  labels = torch.tensor(data.label.values, dtype=torch.lon
g).to(device)
  return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    data_loader = DataLoader(dataset, sampler=data_sampler,
batch_size=batch_size)
    return data_loader
epochs=5
batch_size=32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-c
ased",
    do_lower_case=False
)
train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSamp
ler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, Sequential
Sampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSa
mpler, batch_size)

print(train_dataset[0])
```

- BERT 토크나이저 클래스: 사전 학습된 토크나이저 불러와 전처리 수행
  - 다중 언어 지원
  - 대소문자 유지
  - do_lower_case: 소문자 유지

BERT 모델 선언

```
from torch import optim
from transformers import BertForSequenceClassification


model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-c
ased",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e
-8)
```

- AdamW: 활성화 함수, Adam 최적화 함수에 가중치 감쇠를 추가한 변형된 경사하강법 알고리즘

```
for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children
():
        print("└", sub_name)
        for ssub_name, ssub_module in sub_module.named_chil
dren():
            print("│  └", ssub_name)
            for sssub_name, sssub_module in ssub_module.nam
ed_children():
                print("│  │  └", sssub_name)
```

```
    bert
     └ embeddings
     |    └ word_embeddings
     |    └ position_embeddings
     |    └ token_type_embeddings
     |    └ LayerNorm
     |    └ dropout
     └ encoder
     |    └ layer
     |    |    └ 0
     |    |    └ 1
     |    |    └ 2
     |    |    └ 3
     |    |    └ 4
     |    |    └ 5
     |    |    └ 6
     |    |    └ 7
     |    |    └ 8
     |    |    └ 9
     |    |    └ 10
     |    |    └ 11
     └ pooler
     |    └ dense
     |    └ activation
    dropout
    classifier
```

```
import numpy as np
from torch import nn
import os

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_fl
```

```
at)

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(input_ids=input_ids, attention_mask
=attention_mask, labels=labels)

        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloade
r:
            outputs = model(input_ids=input_ids, attention_
mask=attention_mask, labels=labels)
            logits = outputs.logits

            loss = criterion(logits, labels)
            logits = logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            accuracy = calc_accuracy(logits, label_ids)

            val_loss += loss.item()
```

```
            val_accuracy += accuracy

    val_loss = val_loss/len(dataloader)
    val_accuracy = val_accuracy/len(dataloader)
    return val_loss, val_accuracy


best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_datalo
ader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f}
Val Loss: {val_loss:.4f} Val Accuracy {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        os.makedirs("../models", exist_ok=True)
        torch.save(model.state_dict(), "../models/BertForSe
quenceClassification.pt")
        print("Saved the model weights")
```